ARM Embedded Systems – Detailed Exam Answers

MODULE – 1

---

1. Explain the concept of pipelining in detail.

Introduction

Pipelining is a technique used in processors to increase instruction execution speed by overlapping the execution of multiple instructions. Instead of completing one instruction before starting the next, the processor divides instruction execution into stages and executes several instructions simultaneously.

It is similar to an assembly line in a factory where different stages of work are performed at the same time.

Purpose of Pipelining

- Increases processor throughput
- Improves CPU performance
- Reduces instruction execution time
- Makes better utilization of hardware resources

Basic Pipeline Stages in ARM7

ARM7 uses a 3-stage pipeline:

1. Fetch (F)
2. Decode (D)
3. Execute (E)

1. Fetch Stage

- Instruction is fetched from memory.
- Program Counter (PC) points to instruction address.

2. Decode Stage

- Instruction is decoded.
- Operands are identified.

3. Execute Stage

- ALU operation or memory operation is performed.
- Result is written back.

Working of Pipeline

Clock Cycle	Stage 1	Stage 2	Stage 3
1	Fetch I1	-	-
2	Fetch I2	Decode I1	-
3	Fetch I3	Decode I2	Execute I1
4	Fetch I4	Decode I3	Execute I2

After pipeline fills, one instruction completes every clock cycle.

Pipeline Diagram

```
Instruction 1 : Fetch -> Decode -> Execute
Instruction 2 :          Fetch -> Decode -> Execute
Instruction 3 :                   Fetch -> Decode -> Execute
```

Advantages of Pipelining

1. Increased instruction throughput
2. Better CPU efficiency
3. Faster execution
4. Improved performance without increasing clock speed significantly

---

Disadvantages of Pipelining

1. Pipeline hazards
2. Branch instructions may flush pipeline
3. Increased hardware complexity
4. Data dependency problems

Pipeline Hazards

1. Data Hazard

Occurs when one instruction depends on result of previous instruction.

2. Control Hazard

Occurs due to branch instructions.

3. Structural Hazard

Occurs when hardware resources are insufficient.

Pipeline Flush

Whenever a branch occurs, wrong instructions in pipeline are discarded and correct instructions are loaded. This process is called pipeline flushing.

Conclusion

Pipelining is one of the most important techniques used in ARM processors to improve performance. ARM7 uses a 3-stage pipeline which enables faster instruction execution and efficient processor utilization.

---

2. Briefly describe exceptions, interrupts, and the vector table.

Exceptions

Exceptions are events that interrupt normal program execution and transfer control to a special routine called an exception handler.

Exceptions may occur due to:

- Reset
- Undefined instruction
- Software interrupt
- Hardware interrupt
- Memory faults

Interrupts

Interrupts are signals generated by hardware or software requesting processor attention.

They temporarily stop current execution and execute Interrupt Service Routine (ISR).

Types of Interrupts

1. IRQ (Interrupt Request)

- Normal priority interrupt
- Used for general peripherals

2. FIQ (Fast Interrupt Request)

- High priority interrupt
- Faster response than IRQ
- Used for time-critical applications

ARM Exception Types

Exception	Description
Reset	Occurs during power ON or reset
Undefined Instruction	Invalid instruction execution
SWI	Software Interrupt
Prefetch Abort	Instruction fetch memory fault
Data Abort	Data access memory fault
IRQ	Normal interrupt
FIQ	Fast interrupt

Vector Table

The vector table contains addresses of exception handling routines.

In ARM, vector table starts at address:

- 0x00000000 or
- 0xFFFF0000

ARM Vector Table

Address	Exception
0x00	Reset
0x04	Undefined Instruction
0x08	SWI
0x0C	Prefetch Abort
0x10	Data Abort
0x14	Reserved
0x18	IRQ
0x1C	FIQ

Working of Interrupt Handling

1. Interrupt occurs
2. Processor completes current instruction
3. Current state saved in CPSR
4. PC loads ISR address from vector table
5. ISR executes
6. Control returns to main program

Advantages of Interrupts

- Efficient CPU utilization
- Faster response to events
- Reduces polling overhead

Conclusion

Exceptions and interrupts are essential for handling abnormal conditions and external events in ARM systems. The vector table provides addresses for handling different exceptions efficiently.

---

3. Describe conditional execution and list the different code suffixes.

Conditional Execution

ARM supports conditional execution where instructions execute only if specified condition is true.

This reduces branching and improves performance.

Each ARM instruction contains a condition field.

Benefits

- Reduces branch instructions
- Improves pipeline efficiency
- Reduces code size
- Faster execution

Condition Flags

Condition checking uses CPSR flags:

Flag	Meaning
N	Negative
Z	Zero
C	Carry
V	Overflow

Syntax

```
ADDNE R1, R2, R3
```

Instruction executes only if condition is true.

ARM Condition Code Suffixes

Suffix	Meaning	Condition
EQ	Equal	Z = 1
NE	Not Equal	Z = 0
CS/HS	Carry Set	C = 1
CC/LO	Carry Clear	C = 0
MI	Minus	N = 1
PL	Plus	N = 0
VS	Overflow Set	V = 1
VC	Overflow Clear	V = 0
HI	Higher	C=1 and Z=0
LS	Lower or Same	C=0 or Z=1
GE	Greater or Equal	N=V
LT	Less Than	N≠V
GT	Greater Than	Z=0 and N=V
LE	Less or Equal	Z=1 or N≠V
AL	Always	Always execute

Example

```
CMP R1, R2
ADDEQ R3, R4, R5
```

Addition occurs only if R1 = R2.

Conclusion

Conditional execution is an important ARM feature that reduces branching and improves processor performance.

---

4. Compare and contrast microprocessors and microcontrollers.

Introduction

Microprocessors and microcontrollers are programmable electronic devices used in computing and embedded systems.

Microprocessor

A microprocessor contains only CPU on a chip. External memory and peripherals are required.

Examples:

- Intel Pentium
- AMD Ryzen

Microcontroller

A microcontroller contains:

- CPU
- RAM
- ROM
- Timers
- I/O ports
- ADC all integrated on a single chip.

Examples:

- ARM7
- 8051
- PIC

Comparison Table

Feature	Microprocessor	Microcontroller
Integration	CPU only	CPU + Memory + I/O

Feature	Microprocessor	Microcontroller
Cost	High	Low
Power Consumption	High	Low
Speed	Very high	Moderate
Memory	External	Internal
Application	Computers	Embedded systems
Size	Larger system	Compact
Complexity	High	Low

Applications

Microprocessor

- Personal computers
- Servers
- Workstations

Microcontroller

- Washing machines
- Robots
- Embedded systems
- Automotive systems

Conclusion

Microprocessors are suitable for general-purpose computing while microcontrollers are ideal for embedded applications due to compact size and low power consumption.

---

5. Explain the ARM core data flow model with a diagram.

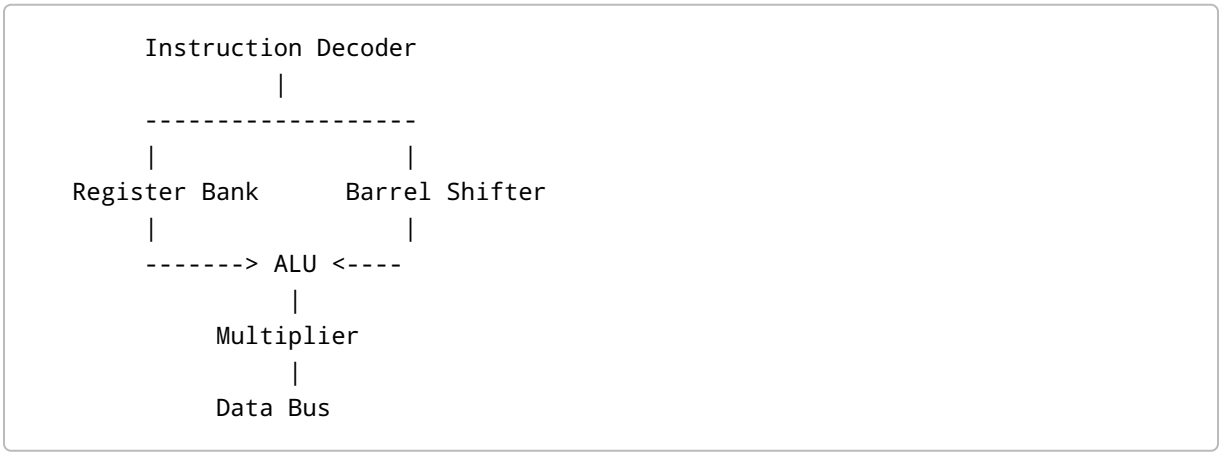
Introduction

ARM core data flow model explains movement of data inside ARM processor during instruction execution.

Main Components

1. Register bank
2. Barrel shifter
3. ALU
4. Multiplier
5. Address register
6. Data bus
7. Instruction decoder

Data Flow Diagram



Working

Register Bank

Stores operands and results.

Barrel Shifter

Performs shift and rotate operations before ALU operation.

ALU

Performs arithmetic and logical operations.

Multiplier

Handles multiplication instructions.

Data Bus

Transfers data between memory and processor.

Features

- Single-cycle execution
- Efficient data processing
- Supports conditional execution
- Fast shifting operations

Conclusion

ARM data flow model provides high performance through efficient interaction between register bank, barrel shifter, ALU, and memory system.

---

6. Explain the four main hardware components of an ARM-based embedded device with a diagram.

Introduction

An ARM-based embedded device consists of several hardware components working together.

Four Main Components

1. ARM Processor Core
2. Memory
3. Input/Output Interfaces
4. Bus System

Block Diagram

```
      -----------------
      | ARM Processor |
      -----------------
              |
    ----------------------
```

```
      |        Bus         |
      ----------------------
       |        |        |
    Memory    I/O     Peripherals
```

---

1. ARM Processor Core

- Executes instructions
- Performs arithmetic and logic operations
- Controls system operation

---

2. Memory

Includes:

- RAM
- ROM
- Flash memory

Stores programs and data.

---

3. Input/Output Interfaces

Connect processor to external devices. Examples:

- UART
- SPI
- GPIO
- USB

---

4. Bus System

Transfers:

- Address
- Data
- Control signals

Types:

- Address bus

- Data bus
- Control bus

Conclusion

These hardware components form the basic architecture of ARM embedded systems and enable efficient processing and communication.

---

7. Explain the different processor modes in ARM7 and provide a schematic of the CPSR with individual bit descriptions.

ARM7 Processor Modes

ARM7 supports multiple processor modes for security and exception handling.

Types of Modes

Mode	Purpose
User	Normal program execution
FIQ	Fast interrupt handling
IRQ	Normal interrupt handling
Supervisor	OS mode after reset
Abort	Memory fault handling
Undefined	Undefined instruction handling
System	Privileged user mode

Features

- Each exception mode has banked registers.
- Improves interrupt response.
- Provides protection and multitasking.

CPSR (Current Program Status Register)

CPSR stores processor status and control information.

CPSR Format

```
 -------------------------------------------------------
|N|Z|C|V|Q|   Reserved   |I|F|T|     Mode Bits         |
 -------------------------------------------------------
```

Bit Description

Bit	Meaning
N	Negative flag
Z	Zero flag
C	Carry flag
V	Overflow flag
Q	Saturation flag
I	IRQ disable
F	FIQ disable
T	Thumb state
Mode bits	Processor operating mode

Mode Bit Values

Mode	Binary
User	10000
FIQ	10001
IRQ	10010
Supervisor	10011

Mode	Binary
Abort	10111
Undefined	11011
System	11111

Conclusion

ARM7 processor modes improve security, interrupt handling, and system reliability. CPSR controls processor status and operating mode.

---

8. Explain pipelining in detail with a schematic.

Definition

Pipelining is a parallel processing technique where multiple instructions are executed simultaneously in different stages.

ARM7 Pipeline Stages

1. Fetch
2. Decode
3. Execute

Schematic

```
Clock 1 : Fetch I1
Clock 2 : Decode I1 | Fetch I2
Clock 3 : Execute I1 | Decode I2 | Fetch I3
Clock 4 : Execute I2 | Decode I3 | Fetch I4
```

Working Principle

- Instructions overlap during execution.
- Processor completes one instruction every cycle after filling pipeline.

Advantages

- High throughput
- Faster execution
- Efficient hardware utilization

Disadvantages

- Branch penalty
- Pipeline hazards
- Complex hardware

Conclusion

ARM pipelining significantly improves processor performance using parallel execution stages.

---

9. Explain major design rules for RISC philosophy and differences between RISC and CISC processors.

RISC Philosophy

RISC stands for Reduced Instruction Set Computer.

RISC processors use simple instructions executed quickly.

Major Design Rules

1. Simple instruction set
2. Fixed instruction length
3. Load/store architecture
4. Large register set
5. Few addressing modes
6. Single-cycle execution
7. Hardwired control unit

Advantages of RISC

- Faster execution
- Easier pipelining
- Reduced hardware complexity
- Better power efficiency

Difference Between RISC and CISC

Feature	RISC	CISC
Instruction Set	Small	Large
Instruction Length	Fixed	Variable
Execution Time	Fast	Slower
Hardware Complexity	Low	High
Addressing Modes	Few	Many
Registers	More	Fewer
Pipelining	Easier	Difficult
Examples	ARM, MIPS	Intel x86

Conclusion

RISC philosophy focuses on simplicity and speed, making ARM processors efficient and suitable for embedded systems.

---

10. Discuss ARM design philosophy and ARM bus technology.

ARM Design Philosophy

ARM processors are designed for:

- High performance
- Low power consumption
- Small code size

- Cost effectiveness

Main Features

1. RISC architecture
2. Load/store design
3. Large register set
4. Conditional execution
5. Pipelining
6. Thumb instruction set

Advantages

- Low power usage
- High speed
- Compact design
- Suitable for portable devices

ARM Bus Technology

ARM uses AMBA (Advanced Microcontroller Bus Architecture).

AMBA Components

1. AHB (Advanced High-performance Bus)

- High-speed communication
- Connects CPU and memory

2. APB (Advanced Peripheral Bus)

- Low-speed peripherals
- Power efficient

3. ASB (Advanced System Bus)

- Older ARM bus architecture

Bus Signals

- Address bus
- Data bus
- Control bus

Advantages of AMBA

- Modular design
- High performance
- Easy integration
- Reduced power consumption

Conclusion

ARM design philosophy combines performance and power efficiency, while AMBA bus technology provides efficient communication between components.

---

11. Explain the ARM core data flow model with a diagram.

Definition

ARM core data flow model represents internal data movement during instruction execution.

Components

- Register bank
- Barrel shifter
- ALU
- Multiplier
- Data bus
- Instruction decoder

Diagram

```
Registers --> Barrel Shifter --> ALU --> Result
                    |
                Multiplier
```

Explanation

- Registers store operands.
- Barrel shifter performs shift operations.
- ALU performs arithmetic and logic operations.
- Multiplier executes multiplication.
- Results are stored back in registers.

Conclusion

ARM data flow architecture improves instruction execution efficiency and processing speed.

---

12. Explain the four main hardware components of an ARM-based embedded device with a diagram, and briefly describe ARM registers used under various modes.

Hardware Components

1. ARM Core

Executes instructions.

2. Memory

Stores data and programs.

3. I/O Devices

Connect external peripherals.

4. Bus System

Transfers data and control signals.

Diagram

```
        ARM Core
            |
        Bus System
      /     |
Memory   I/O   Peripherals
```

ARM Registers

ARM has 37 registers.

General Registers

Register	Purpose
R0-R12	General-purpose
R13	Stack Pointer
R14	Link Register
R15	Program Counter

Special Registers

- CPSR
- SPSR

Banked Registers

Certain modes have separate copies of registers.

FIQ Mode

Has banked:

- R8-R14

IRQ, Supervisor, Abort, Undefined

Have banked:

- R13 and R14

Conclusion

ARM hardware architecture and register organization support efficient multitasking and exception handling.

MODULE – 2

---

1. What are the various logical instructions supported by ARM? Explain each with examples.

Introduction

Logical instructions perform bit manipulation operations.

Logical Instructions

Instruction	Operation
AND	Logical AND
ORR	Logical OR
EOR	Exclusive OR

Instruction	Operation
BIC	Bit Clear
MVN	Move NOT

---

1. AND

Performs bitwise AND.

```
AND R0, R1, R2
```

R0 = R1 AND R2

---

2. ORR

Performs bitwise OR.

```
ORR R0, R1, R2
```

---

3. EOR

Performs Exclusive OR.

```
EOR R0, R1, R2
```

---

4. BIC

Clears selected bits.

```
BIC R0, R1, R2
```

---

5. MVN

Moves complemented value.

```
MVN R0, R1
```

Conclusion

Logical instructions are essential for masking, testing, and manipulating bits in ARM processors.

---

2. Explain the different branching instructions of ARM processor.

Introduction

Branch instructions change execution flow.

Types of Branch Instructions

Instruction	Purpose
B	Branch
BL	Branch with Link
BX	Branch and Exchange
BLX	Branch with Link and Exchange

---

1. B Instruction

Transfers control to another location.

```
B LOOP
```

---

2. BL Instruction

Used for subroutine calls. Stores return address in LR.

```
BL FUNCTION
```

---

3. BX Instruction

Switches between ARM and Thumb state.

```
BX R0
```

---

4. BLX Instruction

Performs subroutine call and state switch.

Conditional Branching

```
BEQ LABEL
BNE LABEL
```

Conclusion

ARM branch instructions support program control, loops, and function calls efficiently.

---

3. Summarize the scheduling of load instructions.

Introduction

Load instruction scheduling improves performance by reducing stalls.

Load Instructions

Examples:

- LDR
- LDM

Scheduling Principles

1. Avoid immediate dependency after load.
2. Insert independent instructions.
3. Reduce pipeline stalls.

Example

```
LDR R0, [R1]
ADD R2, R3, R4
ADD R5, R0, R6
```

Independent instruction placed between load and use.

Benefits

- Better pipeline utilization
- Faster execution
- Reduced delays

Conclusion

Proper scheduling improves ARM processor efficiency.

---

4. Summarize the scheduling of common instruction classes on ARM7TDMI.

Introduction

Instruction scheduling minimizes execution delays.

Common Instruction Classes

1. Data processing
2. Load/store
3. Multiply
4. Branch

Scheduling Guidelines

Data Processing

Usually single-cycle.

Load/Store

Avoid dependent instructions immediately after load.

Multiply

Requires multiple cycles. Schedule independent instructions.

Branch Instructions

Cause pipeline flush. Reduce branch frequency.

Advantages

- Improved throughput
- Reduced pipeline stalls
- Better processor utilization

Conclusion

Efficient scheduling is important for maximizing ARM7TDMI performance.

---

5. Explain the Program Status Register (PSR) in ARM and list its fields. How can PSR be accessed and modified?

Introduction

PSR stores processor status and control information.

Types of PSR

1. CPSR
2. SPSR

Fields of PSR

Field	Description
N	Negative flag
Z	Zero flag
C	Carry flag
V	Overflow flag
Q	Saturation flag
I	IRQ disable
F	FIQ disable
T	Thumb state
Mode bits	Processor mode

Accessing PSR

Read PSR

```
MRS R0, CPSR
```

Modify PSR

```
MSR CPSR_c, R0
```

SPSR

Stores previous CPSR during exception handling.

Conclusion

PSR controls ARM processor state and condition flags.

---

6. Explain how you would use profiling and cycle counting to optimize an ARM assembly program. Give an example.

Introduction

Profiling identifies time-consuming parts of program. Cycle counting measures execution cycles.

Profiling Steps

1. Identify critical code sections.
2. Measure execution time.
3. Optimize bottlenecks.

Cycle Counting

Count CPU cycles required for instructions.

Example:

Instruction	Cycles
ADD	1
MUL	Multiple
LDR	Depends on memory

Optimization Techniques

- Reduce memory accesses
- Use registers efficiently
- Minimize branches
- Schedule instructions properly

Example

Before optimization:

```
LDR R0, [R1]
ADD R2, R0, R3
```

After optimization:

```
LDR R0, [R1]
MOV R4, R5
ADD R2, R0, R3
```

Independent instruction reduces stall.

Conclusion

Profiling and cycle counting help improve ARM assembly performance.

---

7. What is barrel shifter? Describe the various operations barrel shifter supports.

Definition

Barrel shifter is a hardware unit in ARM processor used for shifting and rotating data efficiently.

Features

- Performs shift operation in single cycle
- Works with ALU
- Improves performance

Barrel Shifter Operations

Operation	Description
LSL	Logical Shift Left
LSR	Logical Shift Right
ASR	Arithmetic Shift Right
ROR	Rotate Right
RRX	Rotate Right with Extend

Examples

LSL

```
MOV R0, R1, LSL #2
```

LSR

```
MOV R0, R1, LSR #1
```

ASR

```
MOV R0, R1, ASR #1
```

ROR

```
MOV R0, R1, ROR #4
```

Advantages

- Fast shifting
- Efficient multiplication/division by powers of 2
- Reduces instruction count

Conclusion

Barrel shifter is a powerful ARM feature that enables efficient bit manipulation operations.

---

8. Explain load store instructions with examples.

Introduction

ARM uses load/store architecture. Only load and store instructions access memory.

Load Instructions

Used to read data from memory.

Examples

```
LDR R0, [R1]
```

Loads memory data into R0.

Store Instructions

Used to write data into memory.

Example

```
STR R0, [R1]
```

Stores R0 into memory.

Multiple Load/Store

LDM

Load multiple registers.

```
LDMIA R0!, {R1-R3}
```

STM

Store multiple registers.

```
STMIA R0!, {R1-R3}
```

Addressing Modes

1. Immediate offset
2. Register offset
3. Pre-indexed
4. Post-indexed

Conclusion

Load/store instructions efficiently manage data transfer between memory and registers.

---

9. Explain different data processing instructions in ARM.

Introduction

Data processing instructions perform arithmetic and logical operations.

Types

Arithmetic Instructions

- ADD
- SUB
- ADC
- SBC
- RSB

Logical Instructions

- AND
- ORR
- EOR
- BIC

Compare Instructions

- CMP
- CMN
- TST
- TEQ

Move Instructions

- MOV
- MVN

Examples

ADD

```
ADD R0, R1, R2
```

SUB

```
SUB R0, R1, R2
```

CMP

```
CMP R1, R2
```

MOV

```
MOV R0, #10
```

Features

- Most execute in single cycle
- Can update condition flags
- Support conditional execution

Conclusion

Data processing instructions are fundamental for arithmetic and logical computations in ARM.

---

10. Explain the various multiply instructions of ARM processor.

Introduction

ARM supports efficient multiplication instructions.

Types of Multiply Instructions

Instruction	Purpose
MUL	Multiply
MLA	Multiply Accumulate

Instruction	Purpose
UMULL	Unsigned Multiply Long
UMLAL	Unsigned Multiply Accumulate Long
SMULL	Signed Multiply Long
SMLAL	Signed Multiply Accumulate Long

---

1. MUL

```
MUL R0, R1, R2
```

R0 = R1 × R2

---

2. MLA

```
MLA R0, R1, R2, R3
```

R0 = (R1 × R2) + R3

---

3. UMULL

Produces 64-bit result.

```
UMULL R0, R1, R2, R3
```

---

4. SMULL

Signed multiplication long.

Advantages

- Fast multiplication
- Supports DSP operations
- Efficient arithmetic processing

Conclusion

ARM multiply instructions provide high-speed arithmetic operations for embedded and signal processing applications.